

AMX Kitchen Sink

This is a single-file tour of the implemented AMX language: Markdown documents, expressions, mutation, loops, matching, types, records, pure functions, local modules, data interchange, tables, charts, and report identity. All calculations use synthetic data. The executable tour needs no input mappings and imports only the existing opt-in library beside this file.

Reading convention: exact backtick ``amx`` fences execute. Fences labeled ``text``, ``json``, ``csv``, ``yaml``, ``sh``, or ``markdown`` are literal recipes, not part of the executable program. Invalid examples are explicitly labeled and kept literal. This distinction lets one valid document also explain features that need separate data files, project configuration, or a deliberately failing program.

1. Run, Render, and Export

Run these commands from the repository root. Output parents must already exist. The first command prints the final value environment; the others produce reports or select explicitly exported data. PDF and DOCX are static, offline reports, not interactive HTML or promises of identical pagination.

```
sh bun run src/cli.ts run examples/kitchen-sink.amx bun run src/cli.ts render examples/kitchen-sink.amx --out /tmp/kitchen-sink.html bun run src/cli.ts export pdf examples/kitchen-sink.amx --out /tmp/kitchen-sink.pdf bun run src/cli.ts export docx examples/kitchen-sink.amx --out /tmp/kitchen-sink.docx bun run src/cli.ts run examples/kitchen-sink.amx --output summary=/tmp/kitchen-summary.json --output samples=/tmp/kitchen-samples.csv
```

The HTML report and desktop preview render interactive offline charts. PDF charts are static, but preserve the chart data in their tables, including the shown empty state; DOCX chart behavior is unchanged.

2. Document Structure and Module Imports

The optional YAML frontmatter supplies the document title and report metadata. Historical custom top-level metadata such as ``description`` is inert. Markdown and executable fences retain document order. Imports must come before inputs or any other executable declarations, even when declarations span several fences.

```
import { FailureMode, Risk, Strategy, Asset, MaintenanceTask, LifecycleCost } from "../libraries/asset-management.amx"
```

Imports name explicit exports, not an entire namespace. Paths use forward slashes, start with ``.`` or ``.``, end in lowercase ``.amx``, and must stay inside the entry directory tree after canonicalization. Dependencies run once, before the entry, in their own environments. Imported values are immutable; imports do not re-export them. Cycles, missing exports, name collisions, remote URLs, and packages fail.

Ordinary Markdown Is Still Markdown

Paragraphs can contain **bold**, *emphasis*, ``inline code``, and [local links](../docs/language-spec-v0.3.md).

> Narrative explains assumptions; AMX computes values. Neither turns a synthetic > example into a validated engineering policy.

- An unordered list item. - Another unordered list item with **emphasis**.

1. An ordered list item. 2. Another ordered list item.

| Document part | Role | | --- | | --- | | YAML frontmatter | Metadata and report identity | | Markdown | Narrative and ordinary code listings | | Exact ``amx`` fence | Executable statements | | Inline interpolation | Expression evaluated against the final environment |

The following declaration-looking text does not create a binding:

```
let narrativeOnly = 999
```

```
text let ordinaryFenceOnly = 999
```

```
AMX let wrongCaseFenceOnly = 999
```

```
```amx demo let extraInfoFenceOnly = 999 ```
```

```
~~~amx let tildeFenceOnly = 999 ~~~
```

Four-backtick ordinary fences can display an entire executable-looking block without executing its nested contents:

```
````markdown ```amx let nestedFenceOnly = 999 ``` ````
```

Only exact case-sensitive ``amx`` activates execution. Executable openers permit zero to three leading spaces and at least three backticks; closers need at least the opener's backtick count and no trailing text. The following longer fence is genuinely executable, not just an illustration:

```
let longFenceValue: Number = 4
```

### 3. Literals, Operators, and Precedence

Source numbers include integers and decimals, but not scientific notation (JSON/CSV input numbers support JSON number syntax). A negative number uses unary minus. Expression strings may use double or single quotes; choose the other quote to include a quote character. Backslash escape sequences are not decoded by the expression tokenizer. Identifiers are case-sensitive and may contain digits and underscores after their first letter. There are no implicit coercions in a checked program.

```
let integerValue: Number = 42
let decimalValue: Number = 12.5
let largeValue: Number = 1250
let negativeValue: Number = -8
let stringValue: String = "Pump A"
let quotedValue: String = 'The "pump" is ready'
let singleQuoteValue: String = "The pump's bearing"
let backslashValue: String = "C:\\demo"
let markupValue: String = "<pump> & <script>alert(1)</script>"
let trueValue: Boolean = true
let falseValue: Boolean = false
let nullableNumber: Number? = null
let inferredNumber = 7
let CaseSensitive: Number = 1
let caseSensitive: Number = 2
let asset_01: String = "A-01"
```

```
let addition: Number = 8 + 3
let subtraction: Number = 8 - 3
let multiplication: Number = 8 * 3
let division: Number = 8 / 2
let remainder: Number = 8 % 3
let exponentiation: Number = 2 ^ 3
let rightAssociativePower: Number = 2 ^ 3 ^ 2
let groupedArithmetic: Number = (8 + 3) * 2
let precedence: Number = 2 + 3 * 4
let unaryBeforePower: Number = -2 ^ 2
```

```
let equal: Boolean = 8 == 8
let unequal: Boolean = 8 != 3
let greater: Boolean = 8 > 3
let greaterOrEqual: Boolean = 8 >= 8
let less: Boolean = 3 < 8
let lessOrEqual: Boolean = 3 <= 3
let stringEquality: Boolean = "pump" == "pump"
let booleanEquality: Boolean = true != false
let nullEquality: Boolean = nullableNumber == null
let conjunction: Boolean = true and false
let disjunction: Boolean = false or true
let negation: Boolean = not false
let logicalPrecedence: Boolean = true or false and false
let conditionalValue: String = if greater and negation then "Inspect" else "Monitor"
let lazyConditional: Number = if true then 10 else sqrt(-1)
let evaluatedAnd: Boolean = false and sqrt(0) > 0
let evaluatedOr: Boolean = true or sqrt(0) > 0
```

Power is right-associative: 512. Multiplication binds before addition: 14. AMX retains unary-before-power precedence, so the unary example is 4. Boolean ``and`` binds before ``or``. Conditionals evaluate only the selected branch. The current runtime evaluates both operands of ``and`` and ``or``; do not use them as guards for failing operations.

Markup-like string interpolation inside Markdown inline code: ``<pump> & <script>alert(1)</script>``. Use inline code for this example so its content remains inspectable text. The current report path can interpret markup-like interpolated text outside inline code as raw HTML; interpolation is not an HTML sanitizer. An expression can be interpolated directly: 27.25.

Interpolation is not a declaration or a way to execute surrounding narrative.

## 4. Lists, Ranges, and Every Standard Function

Type suffixes apply left to right. `Number[]` is a list of nullable numbers; `Number[]?` is a nullable list; `Number[][]` is a list of lists. Empty lists need an expected element type in checked AMX. Inclusive integer ranges run in either direction with an implicit step of one, including the equal-bound case.

```
let values: Number[] = [2, 4, 6]
let names: String[] = ["North", "South", "East"]
let flags: Boolean[] = [true, false]
let emptyNumbers: Number[] = []
let nullableItems: Number[]? = [1, null, 3]
let nullableList: Number[]? = null
let nonNullNullableList: Number[]? = [1, 2]
let matrix: Number[][] = [[1, 2], [3, 4]]
let ascending: Number[] = [1 to 3]
let descending: Number[] = [3 to 1]
let singletonRange: Number[] = [2 to 2]
```

```
let total: Number = sum(values)
let emptySum: Number = sum(emptyNumbers)
let minimum: Number = min(values)
let maximum: Number = max(values)
let average: Number = mean(values)
let roundedInteger: Number = round(3.6)
let roundedDecimal: Number = round(3.14159, 2)
let absoluteValue: Number = abs(-8)
let squareRoot: Number = sqrt(81)
let poweredValue: Number = pow(2, 5)
```

Function	Example result	Signature
<code>sum</code>	12	<code>empty list: 0   `Number[] -&gt; Number`     `min`   2   `Number[] -&gt; Number`     `max`   6   `Number[] -&gt; Number`     `mean`   4   `Number[] -&gt; Number`     `round`   4 / 3.14   One number; optional decimal places     `abs`   8   `Number -&gt; Number`     `sqrt`   9   `Number -&gt; Number`     `pow`   32   `(Number, Number) -&gt; Number`</code>

These are the complete current standard functions. `min`, `max`, and `mean` reject empty lists at runtime; `sqrt` rejects negative arguments. AMX has no built-in indexing, string concatenation, list append, date arithmetic, or units.

## 5. Mutable Bindings and Both Loop Forms

`let` creates a binding; a repeated `let` updates it without changing its type. `=` replaces an existing value, and numeric `+=` adds to one. Newlines separate statements; semicolons do not. Braced loops and matches can span lines, but an ordinary expression or constructor cannot continue implicitly across a newline.

```
let mutableTotal: Number = 1
let mutableTotal: Number = 2
mutableTotal = 3
mutableTotal += 4
```

```
let loopTotal: Number = 0
let item: Number = 99
for item in ascending {
  loopTotal += item
}
```

```
let returnCount: Number = 0
let doubled: Number[] = for item in values {
  let scaled: Number = item * 2
  return scaled
  returnCount += 1
}
```

```
let emptyLoopCount: Number = 0
let emptyLoop: Number[] = for value in emptyNumbers {
  emptyLoopCount += 1
  return value
}
```

The updated binding is 7. The statement loop totals 6. Its iterator shadows and restores `item`, which is still 99 afterward. The expression loop collects 4, 8, 12. `return` collects one value per iteration; it is **not** an early exit, so the following statement runs 3 times. The empty loop runs 0 times and produces an empty list.

Each expression loop requires exactly one `return`; statement loops have none. Nested loops, `break`, `continue`, and top-level `return` are unsupported. Loop mutations of pre-existing bindings persist. A binding created only inside a loop is not statically guaranteed to exist after it.

## 6. Match Expressions

Matches support number, string, and boolean literal cases. Exactly one `default` is required; it may appear anywhere. The first matching case wins, including when a case literal repeats. Only the selected expression executes. Cases have no guards, destructuring, null patterns, or expression-valued patterns.

```
let numberMatch: String = match negativeValue {
  case -8 => "Negative eight"
  case 0 => "Zero"
  default => "Other number"
}
let stringMatch: Number = match stringValue {
  default => 0
  case "Pump A" => 10
  case "Pump A" => 20
}
let booleanMatch: String = match trueValue {
  case false => "Disabled"
  case true => "Enabled"
  default => "Fallback"
}
let defaultOnly: Number = match 123 {
  default => 5
}
let lazyMatch: Number = match 1 {
  case 1 => 10
  default => sqrt(-1)
}
```

Results: Negative eight, 10, Enabled, 5, and 10. An early `default` is still a fallback, not a catch-all that skips later cases.

## 7. Record Types, Defaults, Nullability, and DateTime

The four primitives are `Number`, `String`, `Boolean`, and `DateTime`. Declared record types are nominal, closed, and data-only. Even two types with identical fields are distinct. Types must precede their use and cannot be recursive. Fields are immutable and materialized in declaration order.

The presence marker `field?: T` and nullable suffix `T?` are independent. An optional field without a default must be nullable and becomes `null` when omitted. A default permits omission even without a presence marker. Explicit `null` never invokes a default. Defaults are literal scalars, lists, or nested record constructors, not calculations or references to bindings.

```
export type Location {
  site: String = "Unassigned"
  active: Boolean = true
}
```

```
type ExampleRecord {
  id: String
  quantity: Number = 0
  enabled: Boolean = true
  note?: String?
  owner?: String = "Operations"
  reviewedAt?: DateTime? = null
  tags: String[] = ["demo"]
  location: Location = Location { site = "Workshop", active = true }
}
```

```
type EmptyRecord {
}
```

```
let minimalRecord: ExampleRecord = ExampleRecord { id = "A-01" }
let completeRecord: ExampleRecord = ExampleRecord { location = Location { site = "North", active = false }, tags = [], reviewedAt =
"2026-10-03T09:30:00.123456789+02:00", owner = "Reliability", note = "", enabled = false, quantity = 2, id = "A-02" }
let explicitNullRecord: ExampleRecord = ExampleRecord { id = "A-03", note = null, reviewedAt = null }
let emptyRecord: EmptyRecord = EmptyRecord {}
let recordId: String = minimalRecord.id
let nestedSite: String = completeRecord.location.site
let optionalNote: String? = minimalRecord.note
let nullableRecord: ExampleRecord? = minimalRecord
```

```
let safeRecordId: String = if nullableRecord != null then nullableRecord.id else "No record"
let nullableRecords: ExampleRecord?[] = [minimalRecord, null]
```

```
let instant: DateTime = "2026-10-03T07:30:00Z"
let sameInstant: DateTime = "2026-10-03T07:30:00Z"
let offsetInstant: DateTime = "2026-10-03T09:30:00+02:00"
let equalWireTime: Boolean = instant == sameInstant
let differentWireTime: Boolean = instant != offsetInstant
let dateList: DateTime[] = ["2026-10-03T07:30:00Z", "2026-10-04T07:30:00Z"]
```

The minimal record gets its default site **Workshop** and owner **Operations**. Its optional note is null, while the complete record explicitly contains an empty string. A nullable record must be checked before field access; the guarded ID is A-01. Nested defaults are fresh values per construction.

DateTime strings require valid RFC 3339 calendar dates, seconds, and `Z` or a numeric offset; zero through nine fractional digits are allowed. Strings become DateTime only in an expected DateTime context. DateTime equality compares wire strings, not normalized instants: equal wire strings true; different offsets describing the same instant true. There is no date ordering or arithmetic. Exports preserve 2026-10-03T07:30:00Z exactly.

## 8. Pure Functions and Explicit Exports

Functions have typed parameters, a mandatory return type, and one expression body. They may call standard functions or earlier/imported functions and read their own parameters, but cannot capture module bindings, inputs, or iterators. No recursion, forward calls, mutation, statement bodies, or loop expressions.

```
export fn riskProduct(severity: Number, occurrence: Number): Number = severity * occurrence
fn roundedRisk(severity: Number, occurrence: Number): Number = round(riskProduct(severity, occurrence), 2)
fn classify(score: Number): String = if score >= 20 then "Review" else "Monitor"
fn defaultLocation(): Location = Location {}
fn describeSwitch(enabled: Boolean): String = match enabled {
  case true => "Enabled"
  default => "Disabled"
}
fn makeLocation(code: Number): Location = match code {
  case 1 => Location { site = "North" }
  default => Location { site = "South" }
}

let functionScore: Number = roundedRisk(8, 3)
let functionDecision: String = classify(functionScore)
let functionLocation: Location = defaultLocation()
let matchedLocation: Location = makeLocation(1)
let switchDescription: String = describeSwitch(true)
export let exportedScalar: Number = functionScore
exportedScalar += 1
```

The function score is 24, classified as **Review**. An exported binding exposes its **final** value, 25, even after later mutation. This file demonstrates all three public declarations: `export type`, `export fn`, and `export let`. A dependency can import those explicit names, but only entry-module exported values are selectable by `--output`. Functions and types are not data outputs.

## 9. Opt-in Asset Management Library

All six shipped domain schemas are ordinary imported types, not language primitives. They validate shape, not severity scales, identifier policies, engineering formulas, or cross-record relationships.

```
let failureMode: FailureMode = FailureMode { id = "FM-01", name = "Bearing wear", description = "Synthetic observation", severity = 8,
occurrence = 3, detectability = 2 }
let risk: Risk = Risk { id = "R-01", failureMode = failureMode, likelihood = 3, consequence = 8, score = riskProduct(8, 3) }
let strategy: Strategy = Strategy { id = "S-01", name = "Condition monitoring", targetRiskIds = ["R-01"] }
let asset: Asset = Asset { id = "A-01", name = "Workshop pump", assetClass = "Pump", criticality = 4, failureModes = [failureMode] }
let task: MaintenanceTask = MaintenanceTask { id = "T-01", assetId = asset.id, strategyId = strategy.id, title = "Inspect bearing", intervalDays
= 30, estimatedCost = 150 }
let lifecycleCost: LifecycleCost = LifecycleCost { assetId = asset.id, acquisitionCost = 12000, operatingCost = 3000, maintenanceCost =
1500, disposalCost = null }
let knownLifecycleTotal: Number = lifecycleCost.acquisitionCost + lifecycleCost.operatingCost + lifecycleCost.maintenanceCost
```

**Workshop pump** has the illustrative risk score 24 and a known lifecycle subtotal of 16500. Currency and measurement units are author-defined context, not AMX types.

## 10. Typed Data for Tables, Charts, and Outputs

This flat record includes every table scalar type, nullable numeric data, an empty string, and a null string. It is also valid for CSV interchange. The expression loop gives a table enough rows to explore pagination and print flow.

```
export type Sample {
  label: String
  step: Number
  measuredAt: DateTime
  score: Number
  benchmark: Number?
  available: Boolean
  note?: String? = null
  groupName: String
}

export let samples: Sample[] = for step in [1 to 30] {
  return Sample { label = if step % 2 == 0 then "Even" else "Odd", step = step, measuredAt = "2026-10-03T07:30:00Z", score = step * 2,
  benchmark = if step % 5 == 0 then nullableNumber else step * 1.5, available = step % 2 == 0, note = if step % 3 == 0 then optionalNote else "",
  groupName = if step <= 15 then "First half" else "Second half" }
}

type Summary {
  assetId: String
  decision: String
  total: Number
  generatedAt: DateTime
}

let sampleScores: Number[] = for sample in samples {
  return sample.score
}

export let summary: Summary = Summary { assetId = asset.id, decision = functionDecision, total = sum(sampleScores), generatedAt =
instant }
export let exportedNested: ExampleRecord = completeRecord
export let exportedNull: Number? = null
```

The exported summary total is 930. JSON supports scalar, null, record, nested-record, and list exports. CSV supports only a list of one record type whose fields are scalar or nullable scalar; `samples` qualifies, while `exportedNested` does not. Fields are emitted in declaration order. CSV preserves the distinction between an unquoted empty null cell and a quoted empty string. Output names and destinations cannot repeat or conflict with the HTML output. The nullable branches above use previously typed nullable bindings: the current checker does not join a bare `null` branch with a non-null scalar branch.

## 11. Tables and Show-time Snapshots

Declarations define views without emitting them. `show` captures the current source data, and repeated shows produce independent immutable snapshots. The table selects direct scalar fields, not nested paths or list-valued fields.

```
table sampleTable = table(samples) {
  title: "Thirty synthetic observations"
  column label as "Category"
  column step as "Step"
  column measuredAt as "Measured at"
  column score as "Score"
  column benchmark as "Nullable benchmark"
  column available as "Available"
  column note as "Null or empty note"
  column groupName as "Group"
}
show sampleTable

let snapshotRows: Sample[] = [Sample { label = "Original", step = 1, measuredAt = instant, score = 2, benchmark = null, available = true,
groupName = "Demo" }]
table snapshotTable = table(snapshotRows) {
  title: "One view, two snapshots"
  column label as "State at show"
  column score as "Score at show"
}
show snapshotTable
```

## Thirty synthetic observations

Category	Step	Measured at	Score	Nullable benchmark	Available
Odd	1	2026-10-03T07:30:00Z	2	1.5	false
Even	2	2026-10-03T07:30:00Z	4	3	true
Odd	3	2026-10-03T07:30:00Z	6	4.5	false
Even	4	2026-10-03T07:30:00Z	8	6	true
Odd	5	2026-10-03T07:30:00Z	10		false
Even	6	2026-10-03T07:30:00Z	12	9	true
Odd	7	2026-10-03T07:30:00Z	14	10.5	false
Even	8	2026-10-03T07:30:00Z	16	12	true
Odd	9	2026-10-03T07:30:00Z	18	13.5	false
Even	10	2026-10-03T07:30:00Z	20		true
Odd	11	2026-10-03T07:30:00Z	22	16.5	false
Even	12	2026-10-03T07:30:00Z	24	18	true
Odd	13	2026-10-03T07:30:00Z	26	19.5	false
Even	14	2026-10-03T07:30:00Z	28	21	true
Odd	15	2026-10-03T07:30:00Z	30		false
Even	16	2026-10-03T07:30:00Z	32	24	true
Odd	17	2026-10-03T07:30:00Z	34	25.5	false
Even	18	2026-10-03T07:30:00Z	36	27	true
Odd	19	2026-10-03T07:30:00Z	38	28.5	false
Even	20	2026-10-03T07:30:00Z	40		true
Odd	21	2026-10-03T07:30:00Z	42	31.5	false
Even	22	2026-10-03T07:30:00Z	44	33	true
Odd	23	2026-10-03T07:30:00Z	46	34.5	false
Even	24	2026-10-03T07:30:00Z	48	36	true
Odd	25	2026-10-03T07:30:00Z	50		false
Even	26	2026-10-03T07:30:00Z	52	39	true
Odd	27	2026-10-03T07:30:00Z	54	40.5	false
Even	28	2026-10-03T07:30:00Z	56	42	true
Odd	29	2026-10-03T07:30:00Z	58	43.5	false
Even	30	2026-10-03T07:30:00Z	60		true

### One view, two snapshots

State at show	Score at show
Original	2
The preceding snapshot says `Original`, score 2. Yet this narrative already sees the final state, <b>Updated</b> , because all fences run before narrative interpolation. The following fence reuses the earlier view declaration:	
<pre>snapshotRows = [Sample { label = "Updated", step = 1, measuredAt = instant, score = 20, benchmark = null, available = true, groupName = "Demo" }] let finalSnapshotLabel: String = "Updated" show snapshotTable</pre>	

### One view, two snapshots

State at show	Score at show
Updated	20
The earlier snapshot stays unchanged. HTML tables offer stable sort, substring filter, and page sizes 10, 25, and 50 (initially 25); nulls sort last and display as `(null)`. Browser interactions never change AMX values. Print/PDF/DOCX retain every original row in input order, independent of browser table state.	

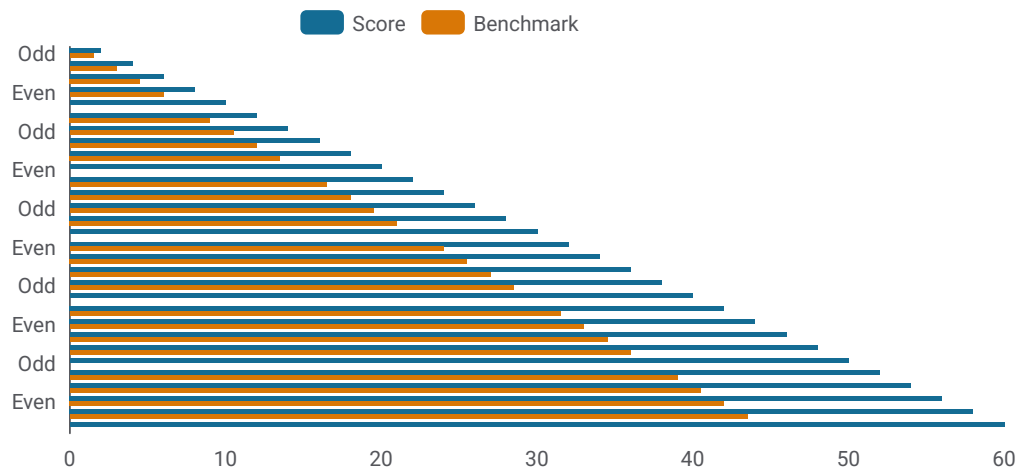
## 12. Every Chart Kind and Data Shape

Every chart requires a title and description. Record bar/column charts use a non-null scalar category; record line charts use a Number or DateTime `x`. Numeric series may be nullable: nulls produce gaps, not fabricated zeroes. Repeated categories remain separate rows. Multiple series retain declared order.

```
chart recordBar = bar(samples) {
title: "Record bar chart"
description: "Repeated categories and two series, including null gaps"
category: label
series score as "Score"
series benchmark as "Benchmark"
}
chart recordColumn = column(samples) {
title: "Record column chart"
description: "Vertical categories in original record order"
category: step
series score as "Score"
}
chart numericLine = line(samples) {
title: "Numeric x line chart"
description: "Points connected in input order, with nullable series gaps"
x: step
series score as "Score"
series benchmark as "Benchmark"
}
chart datetimeLine = line(samples) {
title: "DateTime x line chart"
description: "Preserved wire timestamps; duplicate x values are not aggregated"
x: measuredAt
series score as "Score"
}
chart groupedScatter = scatter(samples) {
title: "Grouped scatter chart"
description: "First-seen group order; null y coordinates omit points"
x: score
y: benchmark
group: groupName
}
chart ungroupedScatter = scatter(samples) {
title: "Ungrouped scatter chart"
description: "A single series with numeric x and y fields"
x: step
y: score
}
show recordBar
show recordColumn
show numericLine
show datetimeLine
show groupedScatter
show ungroupedScatter
```

# Record bar chart

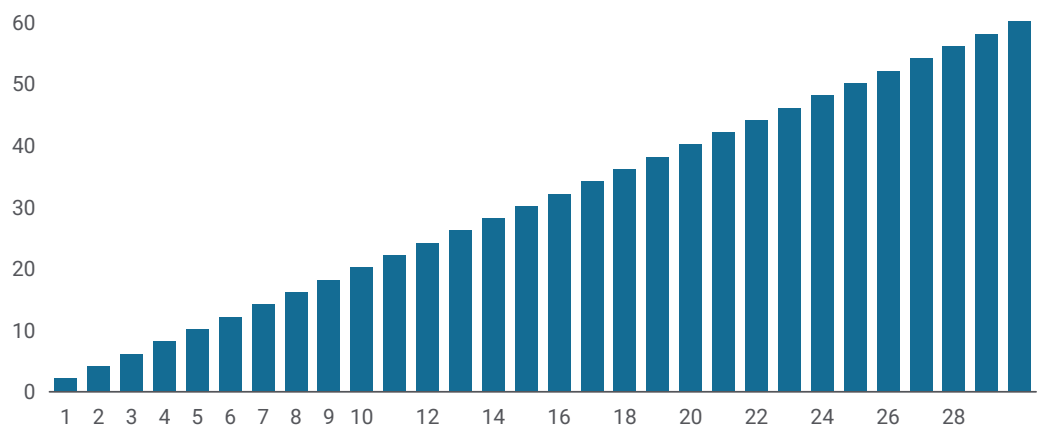
Repeated categories and two series, including null gaps



label	Score	Benchmark
Odd	2	1.5
Even	4	3
Odd	6	4.5
Even	8	6
Odd	10	7.5
Even	12	9
Odd	14	10.5
Even	16	12
Odd	18	13.5
Even	20	15
Odd	22	16.5
Even	24	18
Odd	26	19.5
Even	28	21
Odd	30	22.5
Even	32	24
Odd	34	25.5
Even	36	27
Odd	38	28.5
Even	40	30
Odd	42	31.5
Even	44	33
Odd	46	34.5
Even	48	36
Odd	50	37.5
Even	52	39
Odd	54	40.5
Even	56	42
Odd	58	43.5
Even	60	45

# Record column chart

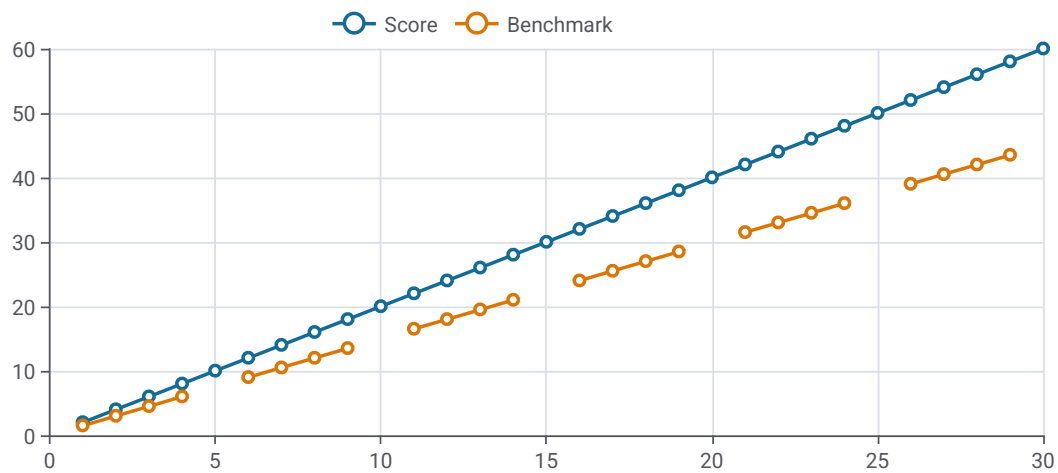
Vertical categories in original record order



label	Score
1	2
2	4
3	6
4	8
5	10
6	12
7	14
8	16
9	18
10	20
11	22
12	24
13	26
14	28
15	30
16	32
17	34
18	36
19	38
20	40
21	42
22	44
23	46
24	48
25	50
26	52
27	54
28	56
29	58
30	60

# Numeric x line chart

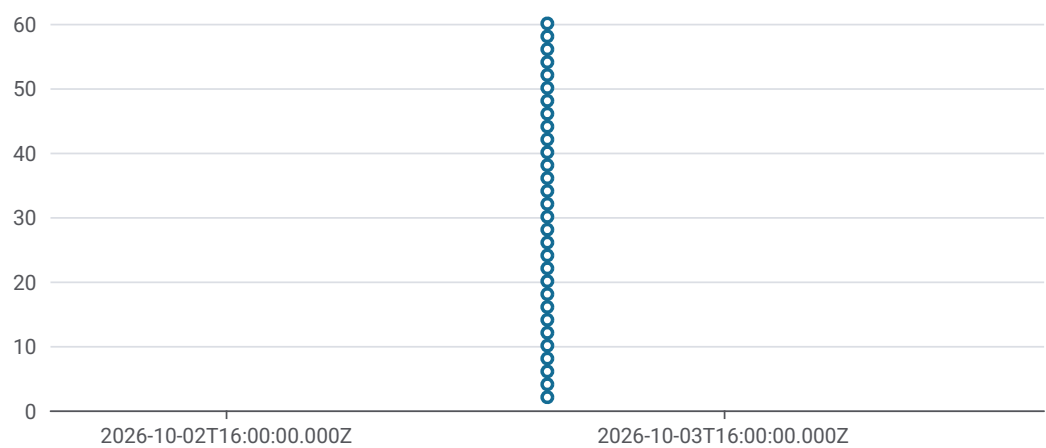
Points connected in input order, with nullable series gaps



x	Score	Benchmark
1	2	1.5
2	4	3
3	6	4.5
4	8	6
5	10	
6	12	9
7	14	10.5
8	16	12
9	18	13.5
10	20	
11	22	16.5
12	24	18
13	26	19.5
14	28	21
15	30	
16	32	24
17	34	25.5
18	36	27
19	38	28.5
20	40	
21	42	31.5
22	44	33
23	46	34.5
24	48	36
25	50	
26	52	39
27	54	40.5
28	56	42
29	58	43.5
30	60	

# DateTime x line chart

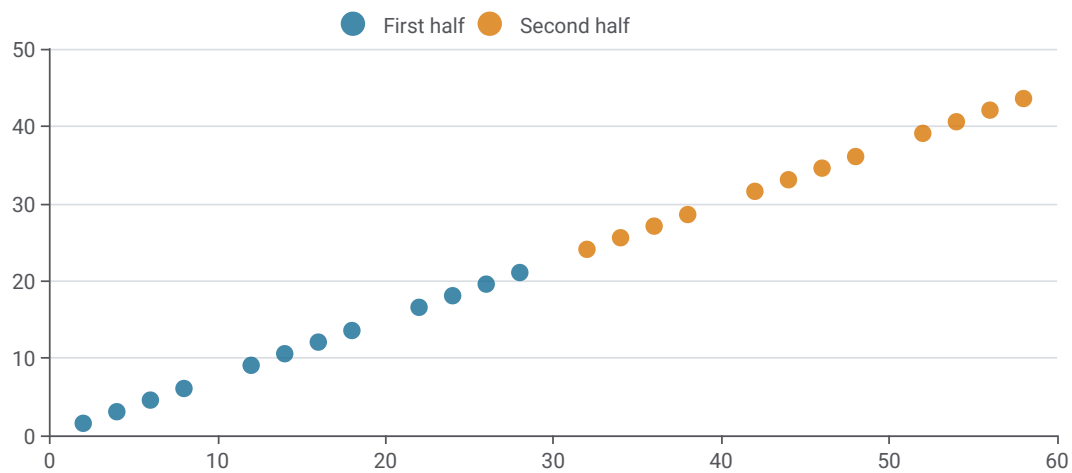
Preserved wire timestamps; duplicate x values are not aggregated



x	Score
2026-10-03T07:30:00Z	2
2026-10-03T07:30:00Z	4
2026-10-03T07:30:00Z	6
2026-10-03T07:30:00Z	8
2026-10-03T07:30:00Z	10
2026-10-03T07:30:00Z	12
2026-10-03T07:30:00Z	14
2026-10-03T07:30:00Z	16
2026-10-03T07:30:00Z	18
2026-10-03T07:30:00Z	20
2026-10-03T07:30:00Z	22
2026-10-03T07:30:00Z	24
2026-10-03T07:30:00Z	26
2026-10-03T07:30:00Z	28
2026-10-03T07:30:00Z	30
2026-10-03T07:30:00Z	32
2026-10-03T07:30:00Z	34
2026-10-03T07:30:00Z	36
2026-10-03T07:30:00Z	38
2026-10-03T07:30:00Z	40
2026-10-03T07:30:00Z	42
2026-10-03T07:30:00Z	44
2026-10-03T07:30:00Z	46
2026-10-03T07:30:00Z	48
2026-10-03T07:30:00Z	50
2026-10-03T07:30:00Z	52
2026-10-03T07:30:00Z	54
2026-10-03T07:30:00Z	56
2026-10-03T07:30:00Z	58
2026-10-03T07:30:00Z	60

# Grouped scatter chart

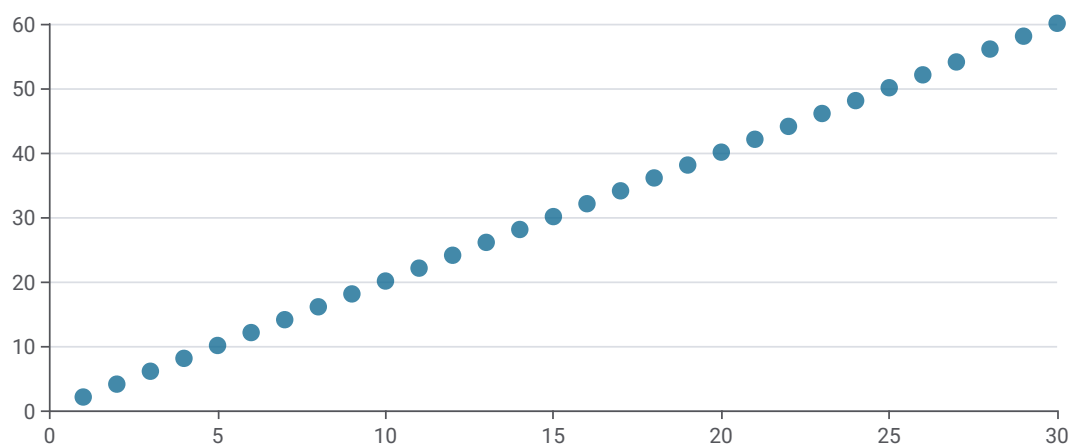
First-seen group order; null y coordinates omit points



x	y	group
2	1.5	First half
4	3	First half
6	4.5	First half
8	6	First half
10		First half
12	9	First half
14	10.5	First half
16	12	First half
18	13.5	First half
20		First half
22	16.5	First half
24	18	First half
26	19.5	First half
28	21	First half
30		First half
32	24	Second half
34	25.5	Second half
36	27	Second half
38	28.5	Second half
40		Second half
42	31.5	Second half
44	33	Second half
46	34.5	Second half
48	36	Second half
50		Second half
52	39	Second half
54	40.5	Second half
56	42	Second half
58	43.5	Second half
60		Second half

# Ungrouped scatter chart

A single series with numeric x and y fields



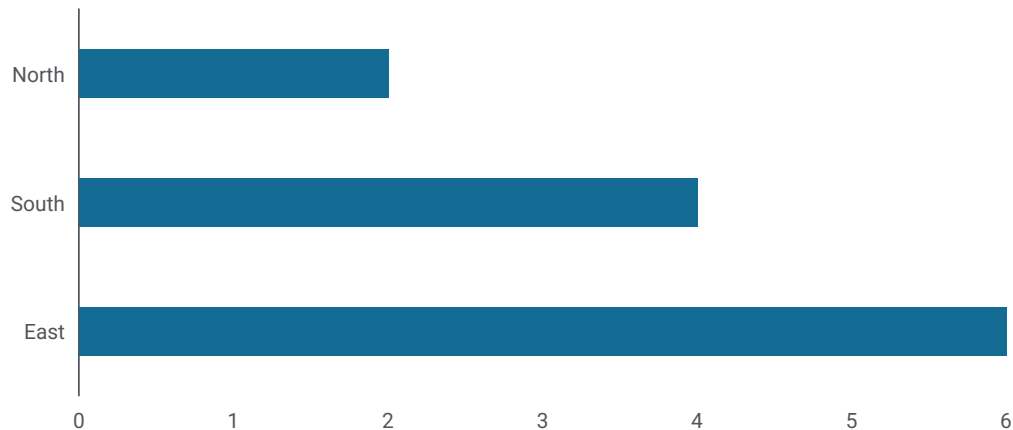
x	y	group
1	2	
2	4	
3	6	
4	8	
5	10	
6	12	
7	14	
8	16	
9	18	
10	20	
11	22	
12	24	
13	26	
14	28	
15	30	
16	32	
17	34	
18	36	
19	38	
20	40	
21	42	
22	44	
23	46	
24	48	
25	50	
26	52	
27	54	
28	56	
29	58	
30	60	

Scalar Number-list bar, column, and line charts have exactly one series label. Optional `labels` must name a String list of the same length; without labels, positions are one-based. Scatter requires records and has no `series`, `labels`, or `category` options.

```
chart scalarBar = bar(values) {
  title: "Scalar bar with labels"
  description: "Three values labeled North, South, and East"
  labels: names
  series "Value"
}
chart scalarColumn = column(values) {
  title: "Scalar column with positional labels"
  description: "Categories default to positions 1, 2, and 3"
  series "Value"
}
chart scalarLine = line(values) {
  title: "Scalar line with labels"
  description: "Numeric positions with author-supplied text labels"
  labels: names
  series "Value"
}
show scalarBar
show scalarColumn
show scalarLine
```

## Scalar bar with labels

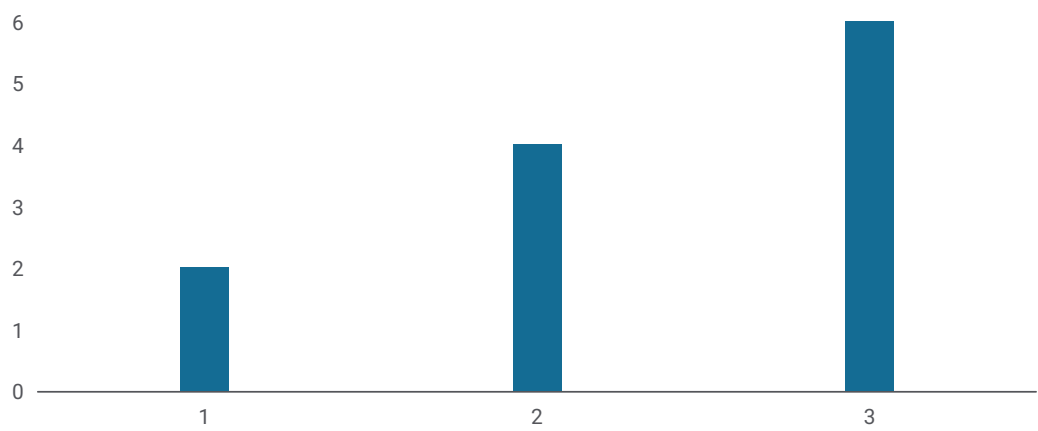
Three values labeled North, South, and East



label	Value
North	2
South	4
East	6

## Scalar column with positional labels

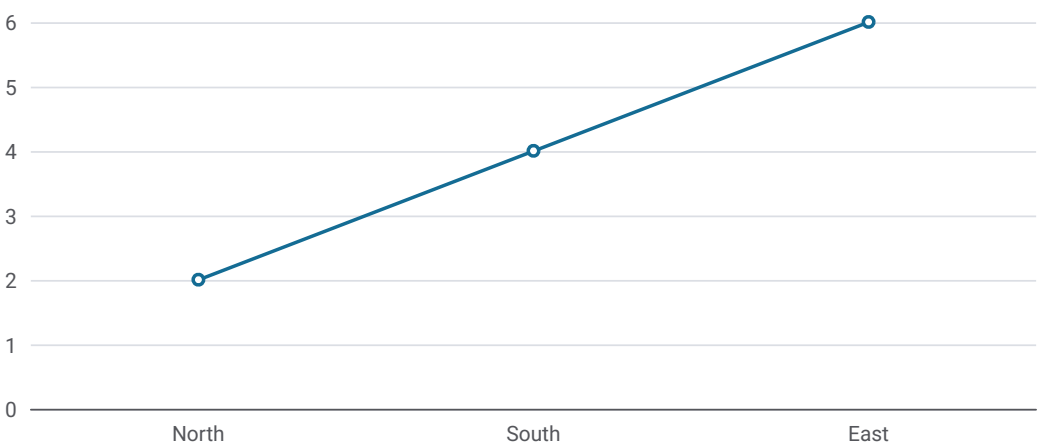
Categories default to positions 1, 2, and 3



label	Value
1	2
2	4
3	6

## Scalar line with labels

Numeric positions with author-supplied text labels



label	Value
North	2
South	4
East	6

Charts include descriptions, legends where applicable, and textual data alternatives. Static reports retain data and labels without interaction. Views are private immutable definitions; they cannot be exported, assigned, passed as ordinary values, declared in dependencies, or shown inside a loop/function.

# Empty Data Is Not an Error

```
let noSamples: Sample[] = []
table emptyTable = table(noSamples) {
  title: "Empty table"
  column label as "Category"
  column score as "Score"
}
chart emptyChart = bar(emptyNumbers) {
  title: "Empty chart"
  description: "No invented rows or plotted values"
  series "Value"
}
show emptyTable
show emptyChart
```

## Empty table

Category	Score
----------	-------

## Empty chart

No invented rows or plotted values

No data

label	Value
-------	-------

The rendered states are `No rows` and `No data`. Declarations with no `show` remain invisible; the following valid view deliberately has no emission:

```
chart unshownChart = column(values) {
  title: "This view is not emitted"
  description: "A definition is not a show statement"
  series "Value"
}
```

## 13. Logical Inputs: Literal Companion Recipes

The standalone HTML comment immediately before this section is AMX's explicit static-report page-break marker: `<!-- page-break -->`. PDF and DOCX start this section on a new page; HTML retains a non-visible comment.

The runnable tour uses local synthetic values so no mappings are required. To explore inputs, use the following **\*\*separate entry-document recipe\*\***. Inputs must follow imports and precede all other executable items. They are immutable, cannot appear in dependencies, and each requires exactly one mapping even if its type is nullable. AMX itself never opens a data path.

```
```text import { Asset } from "./libraries/asset-management.amx" import { Screening } from "./libraries/screening.amx" input asset: Asset input screenings: Screening[] input reviewedAt: DateTime? let name: String = asset.name ```
```

The existing `typed-asset-analysis.amx` implements that workflow with companion JSON/CSV files and named output values. It can be explored without creating another example:

```
```sh bun run src/cli.ts run examples/typed-asset-analysis.amx --input asset=examples/typed-asset.json --input screenings=examples/typed-screenings.csv --input reviewedAt=examples/typed-reviewed-at.json --validation aggregate --output summary=/tmp/typed-summary.json --output resultRows=/tmp/typed-rows.csv bun run src/cli.ts render examples/typed-asset-analysis.amx --input asset=examples/typed-asset.json --input screenings=examples/typed-screenings.csv --input reviewedAt=examples/typed-reviewed-at.json --out /tmp/typed-analysis.html ```
```

Input/output paths are relative to the CLI working directory, not the entry file. Exact lowercase `.json` and `.csv` choose the format. `name=path` splits at the first equals sign; missing, repeated, and unknown mappings fail.

## JSON Shapes and Validation

JSON inputs may be scalars, null, records, or nested lists/records, exactly matching the declared type. Duplicate/unknown keys, missing required fields, non-finite numbers, wrong scalar types, and invalid DateTime strings fail. There is no singleton-to-list conversion or string-to-number coercion. For example, this literal JSON matches `ExampleRecord` above:

```
```json { "id": "A-10", "quantity": 3, "reviewedAt": "2026-10-03T07:30:00Z", "tags": ["demo", "json"], "location": { "site": "South", "active": true } } ```
```

## CSV Shapes and Quoting

CSV inputs require a list of flat records with scalar or nullable scalar fields. Headers are exact field names in any order; optional/defaulted columns may be omitted. The following literal CSV matches `Sample[]`; `groupName` is present because it has no default. It demonstrates an unquoted null, a quoted empty string, commas inside quoted text, and doubled quotes:

```
```csv label,step,measuredAt,score,benchmark,available,note,groupName North,1,2026-10-03T07:30:00Z,2,,true,,Demo "South, workshop",2,2026-10-03T07:30:00Z,4,3,false,,,"Demo "Pump ""C""",3,2026-10-03T07:30:00Z,6,4.5,true,"Quoted observation",Demo ```
```

RFC 4180 quoted cells may contain line breaks. LF or CRLF records and a leading UTF-8 BOM are accepted; bare CR separators, duplicate/unknown/empty headers, wrong row widths, and nested/list fields fail. Number cells use JSON decimal syntax, Boolean cells use lowercase `true`/`false`, and strings preserve text. Exports write LF, one final newline, all fields in declaration order, and RFC 4180 quoting. JSON output is two-space indented with one final newline.

## Aggregate and Fail-fast Validation

These commands intentionally fail, using the existing invalid fixtures:

```
```sh bun run src/cli.ts run examples/typed-asset-analysis.amx --input asset=examples/typed-asset.json --input screenings=examples/typed-invalid-screenings.csv --input reviewedAt=examples/typed-reviewed-at.json --validation aggregate bun run src/cli.ts run examples/typed-asset-analysis.amx --input asset=examples/typed-invalid-asset.json --input screenings=examples/typed-screenings.csv --input reviewedAt=examples/typed-reviewed-at.json --validation fail-fast ```
```

Aggregate is the default and collects independent input failures; fail-fast stops at the first in the same deterministic order. Neither evaluates modules or writes outputs after invalid inputs. Runtime construction and typed value boundaries are also validated; evaluation stops at an invalid computed value.

## 14. Report Identity, Source Visibility, and Project Defaults

The live frontmatter demonstrates every identity field that needs no asset. `title` is a top-level string, not `report.title`. Report text is plain text; status/classification are labels, not access controls. `accent` is a quoted six-digit hex color, used only

where contrast permits, never to recolor data.

The two remaining fields, `logo` and `logoAlt`, require a real local image and an explicitly selected project root. The following is a **configuration recipe**, not a claim that this repository has that image. Place an approved small PNG at the indicated path before selecting this configuration:

```
```json { "version": 1, "inputs": {}, "report": { "organization": "Example Utilities", "logo": "assets/logo.png", "logoAlt": "Example Utilities logo", "accent": "#146C94", "author": "A. Analyst", "status": "Draft", "classification": "Illustrative", "footer": "Synthetic data only", "sourceVisible": true } } ```
```

The filename for that project configuration is `.openamx/project.json`. Desktop input precedence is `per-run > ignored`. `.openamx/local.json` > project defaults; desktop relative paths use the selected project root. Local overrides have only `version` and `inputs`, never report identity. CLI report defaults are read only with an explicit `-project-root`; this does **not** supply CLI input mappings or discover parent projects. `run` does not accept this report flag.

```
```sh bun run src/cli.ts render examples/kitchen-sink.amx --project-root . --out /tmp/kitchen-branded.html bun run src/cli.ts export pdf examples/kitchen-sink.amx --project-root . --out /tmp/kitchen-branded.pdf bun run src/cli.ts export docx examples/kitchen-sink.amx --project-root . --out /tmp/kitchen-branded.docx ```
```

Those project-root recipes require a valid selected project configuration if one is present; they do not install the literal configuration above. Frontmatter overrides project report defaults field by field, then neutral defaults apply. A frontmatter logo override needs its own matching `logoAlt`. Unknown report keys, duplicate keys, invalid types, and empty field strings fail.

For a source-hidden report, replace the live frontmatter setting with this **literal alternative**, keeping other frontmatter fields as needed:

```
```yaml --- title: "AMX Kitchen Sink" report: sourceVisible: false --- ```
```

Hidden source removes listings only; narrative and shown views retain their positions. This is not a confidentiality boundary: original files, inputs, view alternatives, and exports may still disclose data. Set `true` or remove the override to recover the default visible provenance.

Logos accept only contained project-relative PNG/JPG paths, not URLs, SVG, absolute paths, `..`, or symlinks. A logo requires alt text, is at most 256 KiB, and has dimensions at most 1024 by 1024. The trusted caller validates and sanitizes it to embedded PNG; reports do not fetch remote assets. Invalid identity/assets or export preparation fail before changing the destination.

## 15. Diagnostics and Deliberately Invalid Syntax

The following examples are **not executable**. They document constraints you can explore by putting one into a separate exact `amx` fence. Do not turn this whole listing into a live block: each line demonstrates a different failure.

```
```text let badNumber: Number = "3" let missingInitializer: Number let changedType = 1 changedType = "one"
unknownBinding = 1 let semicolon = 1; let second = 2 let badNull: Number = null let badRange = [1.5 to 3] let badDate:
DateTime = "2026-02-30T12:00:00Z" let badConstructor = ExampleRecord { id = "A", extra = true } let duplicateField =
ExampleRecord { id = "A", id = "B" } let missingField = ExampleRecord {} let badField = minimalRecord.unknown let
badEquality = values == values let badCall = sum(1) let runtimeEmptyMean = mean(emptyNumbers) let
runtimeNegativeRoot = sqrt(-1) fn capturesBinding(): Number = integerValue return 1 ```
```

Additional invalid forms include a match without exactly one default, a non-literal case, optional non-nullable fields without defaults, recursive types/functions, dynamic field access, assignment to inputs/imported values, exports of views, nested loops, wrong chart options, non-scalar table columns, and scalar chart labels of the wrong length. An unclosed executable fence is a parse error at its opener. An unclosed ordinary fence remains narrative. There is no executable comment syntax; explanations belong in Markdown outside the executable fence, not in `//` or `#` source comments.

| Diagnostic family | Meaning | | — | | Parser errors | Invalid executable syntax/fences with original source locations | | `AMX3001` | Unknown name, type, or field | | `AMX3002` | Incompatible type | | `AMX3003` | Invalid operator, expression, or view form | | `AMX3004` | Invalid function call | | `AMX3005` | Duplicate or invalid declaration | | `AMX4001` | Input mapping/read failure | | `AMX4002` | Malformed JSON/CSV | | `AMX4003` | Shape, required-field, nullability, conversion, or labels-length failure | | `AMX5001` | Invalid/unavailable module path | | `AMX5002` | Missing export, duplicate name, or collision | | `AMX5003` | Import cycle | | `AMX6001` | Invalid output selection, destination, config, or identity | | `AMX6002` | Serialization, asset-engine, or filesystem write failure |

Source diagnostics use original-document one-based lines and UTF-16 columns, including YAML and fence delimiters. Input diagnostics add logical names and JSON pointers or CSV record/field paths. Parse/link/check errors prevent input loading and evaluation; input errors prevent evaluation; evaluation or report preparation errors prevent writes. Static checking is activated by typed syntax, modules, inputs, or views; untyped V0.2-only documents retain their historical path. Analysis need not enumerate every possible error in one invocation.

PDF/DOCX exports preflight paths and prepare locally before atomic replacement; existing destinations survive pre-rename failures. Named JSON/CSV outputs are serialized before writes, but several output destinations are not a single transaction: an eventual filesystem failure may leave earlier outputs written.

## 16. Formatting and Authoring Surfaces

Canonical formatting touches executable fence contents, not Markdown, YAML, ordinary fences, or delimiter spelling. It uses LF, two-space braced indentation, preserved internal blank lines, no trailing whitespace, and exactly one final newline for nonempty block content. It does not rewrite expression spelling.

This file offers concrete targets for editor highlighting, completion, hover, definition, symbols, and references: imported domain types, local record fields, parameters, shadowed iterators, repeated bindings, functions, and views. Static editor analysis is not execution and does not load JSON/CSV inputs. Run/preview uses the designated entry's current text; opening a dependency does not silently turn it into the entry. Source hiding is report presentation, not editor hiding.

## 17. Coverage Index and Boundaries

| Surface | Sections | | --- | --- | | YAML, Markdown, exact/non-executable/long/nested fences | 2, 14 | | Literals, every operator, precedence, lazy evaluation, interpolation | 3 | | Lists, nullable/nested suffixes, ranges, all nine standard functions | 4 | | Repeated let, assignment, compound assignment, both loops, iterator scope | 5 | | Numeric/string/Boolean/default-only match, duplicate cases, lazy selection | 6 | | Nominal/empty/nested records, defaults, optional/null fields, DateTime | 7 | | Typed/pure/zero-argument/composed functions, record returns, three export forms | 8 | | All six opt-in domain schemas | 9 | | Typed computation and named scalar/null/record/list data outputs | 10 | | Every table scalar, pagination, empty tables, repeated cross-fence snapshots | 11, 12 | | Bar/column/line/scatter, scalar/record sources, labels, groups, gaps, empty/unshown views | 12 | | Inputs, JSON/CSV conversion/quoting, aggregate/fail-fast validation | 13 | | All report fields, project/local defaults, assets, source visibility, HTML/PDF/DOCX | 1, 14 | | Diagnostic families, phase barriers, invalid forms, safe destination writes | 15 | | Formatter and static authoring entry points | 16 |

This tour covers implemented language/report features through V0.5, not future product plans. It does not imply support for enums, inheritance, general unions, constraints, recursion, methods, arbitrary scripting, network access, remote imports, filesystem functions, units/currency, or date arithmetic. The formal contracts remain the V0.2, V0.3, V0.4, and V0.5 specifications in `docs`.